

$$RSS = \frac{1}{2} \sum_{n=1}^N (y_n - \underbrace{w^T x_n}_{\substack{\text{scalair} \\ \text{label de la ligne } n}})^2$$

y_n 1x1 scalair
label de la ligne n
 w $D \times 1$ vecteur
 $\begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_D \end{pmatrix}$ $\rightarrow$ prediction

Prérequis: norme L_2 au norme euclidienne

Soit v vecteur $\|v\|_2 \triangleq \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}$

Donc $\|v\|_2^2 = v_1^2 + v_2^2 + \dots + v_n^2$

ainsi $\|v\|_2^2 \triangleq v^T v$

$$v = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{pmatrix}$$

1×1 par prediction
 x_n $D \times 1$ vecteur
 $\begin{pmatrix} x_{n1} \\ x_{n2} \\ \vdots \\ x_{nD} \end{pmatrix}$ pour prediction

Reprenons donc avec $Xw - y$

X $N \times D$ matrice
 $\begin{pmatrix} x_{11} & x_{12} & \dots & x_{1D} \\ x_{21} & x_{22} & \dots & x_{2D} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \dots & x_{ND} \end{pmatrix}$
 $\begin{pmatrix} x_{11} \\ x_{12} \\ \vdots \\ x_{1D} \end{pmatrix}$ vecteur (voir plus haut)
 $\begin{pmatrix} x_{21} \\ x_{22} \\ \vdots \\ x_{2D} \end{pmatrix}$ vecteur
 $\begin{pmatrix} x_{N1} \\ x_{N2} \\ \vdots \\ x_{ND} \end{pmatrix}$ vecteur
 $\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$ N labels des n rows du df.

$$\|Xw - y\|_2^2 = \underbrace{\begin{pmatrix} x_{11} & x_{12} & \dots & x_{1D} \end{pmatrix}}_{1 \times D} \underbrace{\begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_D \end{pmatrix}}_{D \times 1} - \underbrace{\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}}_{N \times 1}$$

$\Rightarrow$ L'anneau est que cela change la dimension de $\vec{x}$ (car le haut est $D \times 1$ et le bas est $1 \times D$)
 $\Rightarrow$ Utilisons les transposés pour rendre cohérent.

$$\begin{aligned} \|Xw - y\|_2^2 &= \underbrace{\begin{pmatrix} w^T x_1 & w^T x_2 & \dots & w^T x_n \end{pmatrix}}_{1 \times N} \underbrace{\begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}}_{N \times 1} - \underbrace{\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}}_{N \times 1} \\ &= \sum_{n=1}^N (w^T x_n - y_n)^2 \\ &= \sum_{n=1}^N \underbrace{\left(\underbrace{y_n}_{\substack{\text{scalair} \\ 1 \times 1}} - \underbrace{w^T x_n}_{\substack{\text{vecteur} \\ N \times D \times D \times 1}} \right)^2}_{\substack{\text{produit} \\ N \times 1}} \end{aligned}$$

Donc $\frac{1}{2} \|Xw - y\|_2^2 = \frac{1}{2} \sum_{n=1}^N (y_n - w^T x_n)^2$

Par définition de L_2 :

$$\frac{1}{2} \|Xw - y\|_2^2 = \frac{1}{2} (Xw - y)^T (Xw - y)$$

$$\nabla_w \text{RSS}(w) = X^T X w - X^T y \quad ? \text{ PÉRIEVE}$$

ANALOGIE AVEC SCALAIRES

$$\begin{aligned} \text{RSS}(w) &= \frac{1}{2} (Xw - y)^T (Xw - y) \approx \frac{1}{2} (aw - y)^2 \\ \Rightarrow \nabla_w \text{RSS}(w) &\approx \frac{\partial}{\partial w} \left(\frac{1}{2} (aw - y)^2 \right) \approx \frac{1}{2} (a^2 w^2 + y^2 - 2awy) \\ &\approx \frac{1}{2} \cdot 2a^2 w - \frac{1}{2} \cdot 2ay \\ &\approx a^2 w - ay \end{aligned}$$

2 parties : 1) developpement
2) gradient

$$\xrightarrow{\text{SS}} X^T X w - X^T y$$

→ PÉRIEVE A PARTIR de l'éq: $\text{RSS}(w) = \frac{1}{2} (Xw - y)^T (Xw - y)$

PRÉREQUIS 1) $(A-B)^T (A-B) = A^T A - 2A^T B + B^T B$
 $\approx (a-b)^2 = a^2 - 2ab + b^2$

2) $(AB)^T = B^T A^T$ (loi de TRANSPOSITION d'un PRODUIT)

si on développe le RSS
 $\text{RSS}(w) = \frac{1}{2} (Xw - y)^T (Xw - y)$
 $\text{par prérequis 1)} = \frac{1}{2} \underbrace{\left(\frac{Xw}{N \times 1} \right)^T \frac{Xw}{N \times 1}}_{N \times 1} - \underbrace{\frac{1}{2} \frac{y^T Xw}{N \times 1}}_{N \times 1} + \frac{1}{2} \frac{y^T y}{N \times 1}$

$X: N \times D$ mat.
 $w: D \times 1$ vecteur
 $y: N \times 1$ vecteur

par prérequis 2) $= \frac{1}{2} w^T X^T X w - y^T X w + \frac{1}{2} y^T y$

PRÉREQUIS 3) $\nabla_w (b^T w) = b \approx \frac{\partial}{\partial w} aw = a$

PREUVE 1) $b^T w$ - produit scalaire $= \sum_i b_i w_i$

$N \times D \times D \times 1 \rightarrow N \times 1$

1) $\nabla_w (b^T w) = \left[\frac{\partial}{\partial w_1} (b^T w) \right] = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_N \end{bmatrix} = b$

$\frac{\partial}{\partial w_1} (b^T w) = \frac{\partial}{\partial w_1} (b_1 w_1 + b_2 w_2 + \dots + b_N w_N) = b_1$

A) $\nabla_w (w^T A w) = 2Aw$ (si A symétrique) $\approx f(w) = aw^2 \rightarrow f'(w) = 2aw$
 Preuve: Soit $A \in \mathbb{R}^{n \times n}$, $w = \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{pmatrix}$, $A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ \vdots & \vdots \end{pmatrix}$

$f(w) = w^T A w = a_{11} w_1^2 + (a_{12} + a_{21}) w_1 w_2 + a_{22} w_2^2$
 $\frac{\partial f}{\partial w_1} = 2a_{11} w_1 + (a_{12} + a_{21}) w_2$
 $\frac{\partial f}{\partial w_2} = 2a_{12} w_1 + 2a_{22} w_2$

Si A symétrique: $a_{12} = a_{21} \rightarrow \nabla_w f(w) = \begin{pmatrix} 2a_{11} w_1 + 2a_{12} w_2 \\ 2a_{12} w_1 + 2a_{22} w_2 \end{pmatrix}$

Revenons avec la deuxième partie.

Rappel : $RSS(w)$

$$= \frac{1}{2} w^T X^T X w - y^T X w + \frac{1}{2} y^T y$$

$\nabla_w RSS(w)$
Dérivée d'une fonction
scalaire par un vecteur
 $D \times 1$

$$= \nabla_w \left(\frac{1}{2} w^T X^T X w \right)$$

mat 2x2
symétrique

↓
c'est un $D \times 1$

↓ par (4)

$$= \frac{1}{2} \cdot 2 \cdot X^T X w$$

↓ par (3)

$$= X^T X w - y$$

coefficient de w (par (3))

Attention aux dimensions !
→ $-X^T y$ (au lieu de $y^T X$)

$$\nabla_w RSS(w)$$

$D \times 1$

$$= X^T X w - X^T y$$

$D \times 1$

GEOMETRIC INTERPRETATION of LEAST SQUARES

A ce stade nous connaissons la valeur de $\hat{y}$, mais pas son équation, mais ni la connaissance d'ailleurs, mais

Concernant $\hat{y}$, nous connaissons son équation mais pas encore sa valeur. Concernant $\hat{y}$, elle aussi on ne connaît pas encore sa valeur.

"NORMAL EQUATIONS"

$$X^T X \hat{y} = X^T y$$

$$\text{avec } \hat{y} = X^{-1} y$$

inconnue
connue

La solutionnant par ω , on va trouver la valeur de ω qui minimise la RSS (ω) la dérivée et qui minimise la RSS (ω) de $\text{Res}(\omega)$

Prendons l'exemple (très mal fait d'ailleurs) 11, 3 comme base de travail.

Soit $N = 3$, $D = 2$

$$\begin{cases} m = 1 \\ m = 2 \\ m = 3 \end{cases} \begin{cases} x_1 \\ x_2 \\ x_3 \end{cases} \begin{cases} y_1 \\ y_2 \\ y_3 \end{cases}$$

N
lignes

$$\begin{aligned} \hat{y} &= \frac{1}{N \times 1} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} \\ &= \frac{1}{N \times 1} \begin{pmatrix} x_1 y_1 \\ x_2 y_2 \\ x_3 y_3 \end{pmatrix} \\ &= \frac{1}{N \times 1} \begin{pmatrix} x_1 y_1 \\ x_2 y_2 \\ x_3 y_3 \end{pmatrix} \end{aligned}$$

produit scalaire

$$\hat{y} = \frac{1}{N \times 1} \begin{pmatrix} x_1 y_1 \\ x_2 y_2 \\ x_3 y_3 \end{pmatrix} = \frac{1}{N \times 1} \begin{pmatrix} x_1 y_1 \\ x_2 y_2 \\ x_3 y_3 \end{pmatrix}$$

$$\begin{pmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \hat{y}_3 \end{pmatrix} = \begin{pmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} \omega_1 x_1 \\ \omega_2 x_2 \\ \omega_3 x_3 \end{pmatrix}$$

var

var

coeff

$$b = x \cdot y = \omega \cdot x \cdot y$$

3 bases du sous-espace de $\mathbb{R}^3$
3 bases du plan

Re-analysons cette équation:

$$\hat{y} = \omega_1 x_1 + \omega_2 x_2$$

Combinaison linéaire des 2 vecteurs (bases)

→ Tous les vecteurs appartenant au plan (donc espace), et un $\hat{y}$ dans le plan passe du vecteur $\hat{y}$

Visuellement, voir matricielle: projection - sous-espace - région - région

Lecture très importante: L'espace $\mathbb{R}^3$ n'est pas (x_1, x_2, y) (exemple de m)!!! mais composante 1 composante 2 et composante 3. ($m=1, 1 \leq m \leq N$)

Donc $\hat{y} = (y_1, y_2, y_3)$ est un vecteur dans $\mathbb{R}^3$ avec y_1 valeur sur l'axe composante 1, y_2 sur l'axe comp. 2 et y_3 sur l'axe comp. 3.

Donc les vecteurs $\hat{y}_1$ et $\hat{y}_2$ sont des sous-espace (des droites de $\mathbb{R}^3$)

En 2D, on trouve un plan contenant tous les $\hat{y}$ possibles (Cours. linéaire)



$\frac{d}{dx} \log = \frac{1}{x}$

ECRITURES de $L(\theta)$ (LOG-LOSS)

Nomenclature Nig vs Numpy

$\frac{\omega}{N}$ je suis en cellule isolée

$L(\omega) = - \sum_{i=1}^N y_i \log(h(x_i)) + (1-y_i) \log(1-h(x_i))$

avec $h_{\omega}(x) = h(x) = \frac{1}{1+e^{-\omega x}}$ différentiel de la selon class. C'est moi je construis d'une log

$\omega = \theta$ $N \times 1$

$= \sum_{i=1}^N y_i \log(h_{\omega}(x_i)) + (1-y_i) \log(1-h_{\omega}(x_i))$

1-y: variations de la y et log: variations de la y

y $N \times 1$ SIGMOÏDE

$h_{\omega}(x) = \sigma(\omega x)$ 1×1

$h_{\omega}(x) = \sigma(\omega x)$ $N \times 1$ SIGMOÏDE

avec $h_{\omega}(x) = \sigma(x_{\omega}) = \frac{1}{1+e^{-\frac{x_{\omega}}{N \times 1}}}$

DÉRIVÉE de $L(\omega)$ par ω

Composons pour rapel à la régression linéaire

Réglin: $h_{\omega}(x) = \omega_0 + \omega_1 x_1 + \omega_2 x_2 + \dots + \omega_N x_N$ ($\omega_1 x_1 + \omega_2 x_2 + \dots + \omega_N x_N$)

$h_{\omega}(x) = \sum_{i=1}^N \omega_i x_i = \omega^T x$

LogReg: $h_{\omega}(x) = \frac{1}{1+e^{-\omega x}}$

DONC: Au lieu d'avoir une fonction de ω , nous avons une composition de fonctions: $\theta \xrightarrow{z} \omega \xrightarrow{h} L$

On doit appliquer la chaîne de composition

$\nabla_{\omega} L(\omega) = \frac{dL}{d\omega} = \frac{dL}{dh} \cdot \frac{dh}{dz} \cdot \frac{dz}{d\omega}$

$L(h) = - \sum_{i=1}^N y_i \log(h) + (1-y_i) \log(1-h)$

$\frac{d}{dz} \ln(x) = \frac{1}{x}$

donc ce ne sont pas vraiment des logs -

$\frac{dL}{dh} = - \sum_{i=1}^N y_i \frac{1}{h} + \sum_{i=1}^N (1-y_i) \frac{1}{1-h}$

Dimension matrices

éléments de la log-loss pour rapel au sujet à la régression linéaire

division éléments par élément: HADAMARD

$= - \sum_{i=1}^N \frac{y_i}{h} + \sum_{i=1}^N \frac{1-y_i}{1-h}$

$$\textcircled{2} \frac{dh}{dz}$$

$$h = \frac{1}{1+e^{-z}} = \sigma(z) = \sigma(Xw) = \sigma(w^T x)$$

$$\text{Posons } u(z) = 1+e^{-z}$$

$$h = \frac{1}{u(z)}$$

$$\Rightarrow \frac{\partial h}{\partial z} = \frac{\partial}{\partial z} \left(\frac{1}{u(z)} \right) = - \frac{u'(z)}{u^2(z)}$$

$$\text{dérivée sachant : } -e^{-z}$$

$$\text{avec } u'(z) = -e^{-z}$$

$$\text{on } h = \frac{1}{(1+e^{-z})}$$

$$= - \frac{-e^{-z}}{(1+e^{-z})^2} = \frac{e^{-z}}{(1+e^{-z})^2}$$

$$= h \cdot \frac{e^{-z}}{(1+e^{-z})}$$

$$= h(1-h)$$

$$1-h = 1 - \frac{1}{1+e^{-z}} = \frac{1+e^{-z}-1}{1+e^{-z}} = \frac{e^{-z}}{1+e^{-z}}$$

Ceci dit par le développement des éléments individuels du Jacobien

En réalité, $\frac{dh}{dz} \sim N \times 1$ = Jacobien $N \times N$.

$$\text{Ainsi, si } h = \begin{bmatrix} h_1 \\ h_2 \\ h_3 \\ h_4 \end{bmatrix}_{N \times 1}$$

$$= \begin{bmatrix} \frac{\partial h_1}{\partial z_1} \\ \frac{\partial h_2}{\partial z_1} \\ \frac{\partial h_3}{\partial z_1} \\ \frac{\partial h_4}{\partial z_1} \end{bmatrix}$$

Alors

$$\frac{\partial h}{\partial z} =$$

$$\begin{bmatrix} \frac{\partial h_1}{\partial z_1} & \frac{\partial h_1}{\partial z_2} & \frac{\partial h_1}{\partial z_3} & \frac{\partial h_1}{\partial z_4} \\ \frac{\partial h_2}{\partial z_1} & \frac{\partial h_2}{\partial z_2} & \frac{\partial h_2}{\partial z_3} & \frac{\partial h_2}{\partial z_4} \\ \frac{\partial h_3}{\partial z_1} & \frac{\partial h_3}{\partial z_2} & \frac{\partial h_3}{\partial z_3} & \frac{\partial h_3}{\partial z_4} \\ \frac{\partial h_4}{\partial z_1} & \frac{\partial h_4}{\partial z_2} & \frac{\partial h_4}{\partial z_3} & \frac{\partial h_4}{\partial z_4} \end{bmatrix}$$

Mais comme $h_i = \sigma(z_i)$ dépend uniquement de z_i :

$$\frac{\partial h_i}{\partial z_i} = \left\{ \frac{\partial}{\partial z_i} \sigma(z_i) = h_i(1-h_i) \right\}$$

$$i \neq j$$

$$\frac{\partial h}{\partial z} = \begin{bmatrix} h_1(1-h_1) & 0 & 0 & 0 \\ 0 & h_2(1-h_2) & 0 & 0 \\ 0 & 0 & h_3(1-h_3) & 0 \\ 0 & 0 & 0 & h_4(1-h_4) \end{bmatrix}$$

$$\left(\frac{\partial h}{\partial z} \right)_{ij} = \delta_{ij} h_i(1-h_i) \text{ avec } \delta_{ij} = \begin{cases} 1 & i=j \\ 0 & \text{sinon} \end{cases}$$

De même, pour $\frac{\partial h}{\partial z}$ avec $h = h_w(x)$ et $z = xw$

$$\frac{\partial h}{\partial z} = \text{diag}(h \odot (1-h))$$

$$h \odot (1-h) \text{ est le vecteur d'éléments du Jacobien, comme on attend de}$$



élément par élément. Quand on change les éléments, seules les diagonales sont affectées. Pour une attention, des

Développement selon Andrew Ng : $\frac{\partial \mathcal{L}}{\partial \theta_j}$

$$\frac{\partial}{\partial \theta_j} \mathcal{L}(\theta) = \left(y \frac{1}{g(\theta^T x)} - (1-y) \frac{1}{1-g(\theta^T x)} \right) \left(\frac{\partial}{\partial \theta_j} g(\theta^T x) \right)$$

$$= \left(\dots \right) g(\theta^T x) (1-g(\theta^T x)) \left(\frac{\partial}{\partial \theta_j} g(\theta^T x) \right) \cdot \frac{1}{g(\theta^T x) (1-g(\theta^T x))}$$

$$= (y - h_\theta(x)) x_j$$

Gradient scalaire
(variation différentielle
de même gradient
vectorel)

(1×1)

$(y - h_\theta(x)) x_j$
(scalaire) (1×1) (scalaire)

un élément de

$\frac{\partial \mathcal{L}}{\partial \theta}$ ou $\frac{\partial \mathcal{L}}{\partial \omega}$ (Grad vectoriel)

ici, c'est $\frac{\partial \mathcal{L}}{\partial \theta_j}$ ou $\frac{\partial \mathcal{L}}{\partial \omega_j}$ (Gradient scalaire)

THE EXPONENTIAL FAMILY

famille

La régression linéaire et logistique ne sont que 2 cas particuliers d'une

(grande) famille de modèles, la famille **EXPONENTIELLE**

$$p(y; \eta) = b(y) e^{\eta^T T(y) - a(\eta)}$$

$a(\eta)$

log partition function ou fonction de normalisation logarithmique.

elle fait en sorte que la probabilité soit bien normalisée, c-à-d.

$$\sum_y p(y; \eta) = 1 \quad \text{ou} \quad \int p(y; \eta) dy = 1$$

(discret) (continu)

- Ex. dans la loi de Bernoulli $a(\eta) = \log(1 + e^\eta)$, a qui arrive que $p(y=0) + p(y=1) = 1$

- Relations:

$$a) E[T(y)] = \nabla_{\eta} a(\eta)$$

($\frac{\partial}{\partial \eta}$)

$$b) \text{Var}(T(y)) = \nabla^2 a(\eta)$$

($\frac{\partial^2}{\partial \eta^2}$)

la matrice de variance/covariance de $T(y)$ est égale à la matrice hessienne (matrice des dérivées secondes) de $a(\eta)$ (par rapport à η)

$T(y)$ - Statistique suffisante de y souvent $T(y) = y$

- Dans certains cas $T(y) \propto y$

a) Distribution Γ (gamma)

$$p(y; \eta) = e^{\eta^T T(y) - a(\eta)}$$

cause: $T(y) = \text{valeur naturelle}$

avec $T(y) = \begin{bmatrix} \log y \\ y \end{bmatrix}$ ici ($\eta = \begin{bmatrix} \eta_1 \\ \eta_2 \end{bmatrix}$)

b) Multinomiale (multinomiale à sens)

on considère souvent y en ONE-Hot vector

$$T(y) = \begin{bmatrix} 1 & y = 1 & \dots & 1 & y = K-1 \end{bmatrix}$$

$T(y)$ = vecteur (individualité) (par ex. en)

KNN (K-Nearest neighbors) [Exemple - based Method]

Définition mathématique

$$p(y=c | \vec{x}, D) = \frac{1}{K} \sum_{m \in N_K(\vec{x}, D)}$$

probabilité conditionnelle

Nombre de voisins (Paramètre à définir)

$$\vec{x} : \text{Dim} \times 1$$

$$D : \frac{N}{I} \text{ exemple}$$

$$D = \{(\vec{x}_1, y_1), (\vec{x}_2, y_2), \dots, (\vec{x}_N, y_N)\}$$

avec $x_i \in \mathbb{R}^{Dim}$
 $y_i \in \{1, \dots, K\}$

$$\text{donc } \frac{N}{I} \times (Dim + 1)$$

$$N_K(\vec{x}, D) = \{1, 2, \dots, N\}^K$$

- Liste des indices des K voisins les plus proches de l'observation de classe D en indices sont autres compris entre 1 et N, et on en a exactement K
- Exemple: D contient 6 points et tu prends $K=3$ alors $N_K(\vec{x}, D) \in \{1, 2, 3, 4, 5, 6\}^3$

indices des vecteurs d'entrée (i des $\vec{x}_i$)

Cela signifie par exemple:

$$N_K(\vec{x}_6, D) = (2, 4, 5)$$

indices des 3 points les plus proches de $\vec{x}_6$

- $K \times 1$ (vecteur de longueur K)

Exemple:

Sont l'image $\vec{m}$ (vecteur de pixels), on choisit $K=3$ (3 voisins les plus proches). Les classes possibles sont chat, chien, lion. Les voisins sont étiquetés [chat, chien, chat].

Alors

$$p(y = \text{chat} | \vec{x}, D) = \frac{2}{3} \quad p(y = \text{chien} | \vec{x}, D) = \frac{1}{3}$$

→ classe prédite = chat.

Paramètres:

1. Paramètres: K (Nombre de voisins)
2. Paramètres non réglables: $d(\vec{x}, \vec{x}_i)$ / distance métrique entre vecteurs d'entrée $\vec{x}$ et $\vec{x}_i$

$$d_H(\vec{x}, \vec{x}_i) = \sqrt{(\vec{x} - \vec{x}_i)^T H (\vec{x} - \vec{x}_i)}$$

Si $H = I$ (matrice identité) → Euclidien
Distance (exemple)

Si $K \neq 1$ Les voisins sont des voisins voisins engendrés $N \rightarrow \infty$

(rien à voir avec la smoothing des données qui transforme les données (pixels ou points) en 1 gradient (plus de lecture donc)

$$\frac{1}{I} (y_m = c)$$

La proba que le label y soit égal à la classe c , sachant un vecteur d'entrée $\vec{x}$ et un ensemble de données D , est égale à la proportion des K plus proches voisins de $\vec{x}$ dans D qui appartiennent à la classe c .

Fictif on indique $\rightarrow$ vaut à c : le voisin m appartenant à la classe c , classe 0

$$(\vec{x}_N, y_N)$$

indices des classes prises par le label

Paramètres (suite)

K : Si K faible (nombre $K=1$)

- Frontière de décision compliquée / complexe, training error = 0
- Risque d'Overfitting ($K=1$)

Si K élevé

- Frontière de décision plus lisse, training error ↑
- Risque d'Underfitting

LEARNING DISTANCE METRICS

Contexte

Imaginons un espace 2D, $\vec{x} \in \mathbb{R}^2$ then speed of trucks
On considère un vecteur (comme une moyenne empirique),

$$\vec{\mu} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Et 2 points,

$$\vec{x}_1 = \begin{bmatrix} 2 \\ 2 \end{bmatrix}, \quad \vec{x}_2 = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

Datasets:

x_1	σ_{x_1}
2	2
x_2	σ_{x_2}
2	0,2

Supposons que la variance est très différente (et les 2 axes)
- feature 1 ($\vec{x}_1$, axe horizontal) varie **fort** (variance = 2)
- feature 2 ($\vec{x}_2$, axe vertical) varie **peu** (variance = 0,04)

Donc la matrice de covariance approximative est :

$$\Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 0,01 \end{bmatrix} \Rightarrow M = \Sigma^{-1} = \begin{bmatrix} 1 & 0 \\ 0 & 100 \end{bmatrix}$$

CALCUL DES DISTANCES

1) DISTANCE EUCLIDIENNE

$$d_E(\vec{x}_1, \vec{x}_2) = \sqrt{(2-0)^2 + (2-0)^2} = \sqrt{8} = 2,83$$

$$d_E(\vec{x}_1, \vec{x}_2) = \sqrt{(2-0)^2 + (0,01-0)^2} = \sqrt{4,01} = 2,01$$

2) DISTANCE DE MAHALANOBIS

$$d_M(\vec{x}_1, \vec{x}_2) = \sqrt{(\vec{x}_1 - \vec{\mu})^T M (\vec{x}_1 - \vec{\mu})} = \sqrt{\begin{bmatrix} 2 & 2 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 100 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \end{bmatrix}} = \sqrt{2 \cdot 1 + 2 \cdot 100} = \sqrt{202} = 14,21$$

$$d_M(\vec{x}_2, \vec{x}_2) = 2,83$$

TABLEAU DES DISTANCES

	Eucl.	Mathélabois
$\vec{x}_1$	2,83	20,1
$\vec{x}_2$	2,01	2,83

matrices des vecteurs
"distances" en
données géométriques

$\Rightarrow$ On préfère la distance de MAHALANOBIS

$$d_M(\vec{x}, \vec{\mu}) = \sqrt{(\vec{x} - \vec{\mu})^T M (\vec{x} - \vec{\mu})}$$

Matrice définie positive

Si $M = I$ (matrice identité), on revient à la distance euclidienne.

En Euclidien : un cercle - tous les points à égale distance

En Mathélabois : le "cercle" est une ellipse dont la forme dépend des variances / covariances (ellipse élargie dans l'axe des variances à grande variance)



COMMENT TROUVER M ? MÉTHODES LINÉAIRES ET CONVEXES

(vs Deep, Non-convex)

Notion de similarité:

La similarité entre 2 vecteurs d'entrée $\vec{x}$ et $\vec{x}'$ est définie par la distance "euclidienne" entre ces 2 vecteurs

$$\Delta(\vec{x}, \vec{x}') = d_H(\vec{x}, \vec{x}') = \sqrt{(\vec{x} - \vec{x}')^T M (\vec{x} - \vec{x}')}$$

MAHALANOBIS

Nous pouvons désormais approcher la matrice de distance de MAHALANOBIS M 2 méthodes: 1) L M N N (Large Margin Nearest Neighbor) $O(N^2)$

- Méthode linéaire, basée sur optimisation convexe
- Rapprache les points de vi classe, impose une marge avec les points des autres classes
- Apprend $M (= W^T W)$

M n'est pas dans le cadre d'une distance, elle doit être

une matrice SYMÉTRIQUE DÉFINIE POSITIVE

Et la condition est qu'il $\exists W$ tel que $M = W^T W$

2) N C A (Neighborhood Component Analysis)

- Méthode linéaire, non statique par optimisation convexe.
- Apprend W et en déduit M via $M = W^T W$

① LMNN (LARGE MARGIN NEAREST NEIGHBOR)

Fonction de coût: $\mathcal{L}(M) = (1 - \lambda) \mathcal{L}_{\text{space}}(M) + \lambda \mathcal{L}_{\text{margin}}(M)$

matrice à apprendre

Fonction des voisins

des points de vi classe

(on ajuste M)

Fonction qui évalue les voisins des classes différentes

(on ajuste M)

et λ hyperparamètre permettant de donner plus d'importance à l'attraction (pull) ou à la répulsion (push)

$$\mathcal{L}_{\text{space}}(M) = \sum_{i=1}^N \sum_{j \in N_i} d_H(\vec{x}_i, \vec{x}_j)^2$$

$\vec{x}_i, \vec{x}_j$: vecteurs d'entrée, $\vec{x}_j$: un des N_i voisins de $\vec{x}_i$ (vecteur négatif)

$\rightarrow$ Cette loss fonction optimise M de sorte que on minimise la distance entre chaque $\vec{x}_i$ et tous les TARGET NEIGHBORS $\vec{x}_j$

$$\mathcal{L}_{\text{margin}}(M) = \sum_{i=1}^N \sum_{j \in N_i} \sum_{l=1}^L \mathbb{1}(y_i \neq y_l)$$

Complexité temporelle $O(N^3)$!

A la fois Δ et $\mathcal{L}_{\text{margin}}$

"HINGE LOSS FUNCTION" (Relevé par un)

$\vec{x}_i$: target négatif (on classe)

$\vec{x}_l$: impostor (classe différente)

m : marge par bien séparer les classes (soient $= 1$)

$$d_H(\vec{x}_i, \vec{x}_l) = \sqrt{(\vec{x}_i - \vec{x}_l)^T M (\vec{x}_i - \vec{x}_l)} \quad \text{si } M = W^T W$$

$$= \sqrt{(\vec{x}_i - \vec{x}_l)^T W^T W (\vec{x}_i - \vec{x}_l)}$$

$$= \|W(\vec{x}_i - \vec{x}_l)\|_2 \quad \text{Donc } d_H(\vec{x}_i, \vec{x}_l) = \|W(\vec{x}_i - \vec{x}_l)\|_2$$

Noter L_2

voir NCA

2) N C A (NEIGHBORHOOD COMPONENT ANALYSIS)

Ici, on va définir une Loss function qu'on va optimiser par Gradient Descent.
 Donc, il faut que la fonction de probabilité de $\vec{x}_i$ ne soit plus élevée (englobe un voisin de plus $\rightarrow$ intervalle de 0,4 à 0,6 par exemple)

$\Rightarrow$ on va devoir créer une fonction softmax linéairement

$$P_{ij}^W = \frac{\exp(-\|W\vec{x}_i - W\vec{x}_j\|_2^2)}{\sum_{l \neq i} \exp(-\|W\vec{x}_i - W\vec{x}_l\|_2^2)} = \frac{e^{-\frac{1}{2}H(\vec{x}_i, \vec{x}_j)}}{\sum_{l \neq i} e^{-\frac{1}{2}H(\vec{x}_i, \vec{x}_l)}}$$

Probabilité
que $\vec{x}_i$ est au $\vec{x}_j$
dans son voisin

où H la distance euclidienne normalisée
(cartésienne)

Rappel: La Softmax transforme des scores en probabilités (en classification)

$$\text{softmax}(s_j) = \frac{e^{s_j}}{\sum_k e^{s_k}}$$

Plus le score s_j est élevé
plus la probabilité (d'être s_j) est grande

ici, on applique softmax à des scores négatifs

$$P_{ij}^W = \frac{\exp(-\|W\vec{x}_i - W\vec{x}_j\|_2^2)}{\sum_{l \neq i} \exp(-\|W\vec{x}_i - W\vec{x}_l\|_2^2)}$$

Plus la distance ($\vec{x}_i - \vec{x}_j$ ou $\vec{x}_i - \vec{x}_l$) est petite, plus l'exponentielle est grande (expo du numérateur ou du dénominateur)

$\rightarrow P_{ij}^W$ est un équilibre de 2 classes

tant que
on connaît les

MAIS! A la grande différence de la probabilité de KNN classique, ici les bornes de chaque $\vec{x}_i$ de données dans l'espace séparant les $\vec{x}_j$!

Il y a un minimum de probabilités (au lieu de pour d'une zone de l'autre)

$\rightarrow$ Cette fonction est dérivable (et lisse) $\rightarrow$ on peut GRADIENT DESCENT.

A partir de maintenant, on peut définir l'objectif de classification supervisée.

$$J(W) = \sum_{i=1}^N \sum_{j \neq i}^N y_i \cdot y_j$$

Et la Loss à minimiser

$$J(W) = 1 - \frac{J(W)}{N}$$

$\rightarrow$ Gradient Descent

(Erreur de Leave-One-Out (LOO))

$\Rightarrow$ On attend W

COMMENT FAIRE POUR RÉALISER LE KNN SANS CALCULER M

= DEEP METRIC LEARNING (alt. aux méthodes linéaires et convexes d'apprentissage de K)

But:

Apprendre une fonction d'EMBEDDING (intégration) $\vec{e} = f(\vec{x}, \vec{\theta})$ qui transforme un input $\vec{x}$ (saussant complexe, comme une image au départ) en un vecteur dans un type de représentation $\vec{e}$.

- les obj. de similarité sont proches (de départ $\vec{x}_i$)
- les obj. de différence sont éloignés (de départ $\vec{x}_i$)

$\Rightarrow$ les distances ont plus de "sens" et $\vec{e}$ sont plus proches ou plus éloignés

Comment: Au lieu d'utiliser la distance de HAYALANOBIS, on va utiliser la DISTANCE EUCLIDIENNE NORMALISÉE

Donc soit $\vec{e} = f(\vec{x}, \vec{\theta}) \in \mathbb{R}^D$ $L < D$ (on entre des features)

Embedding: vecteur de dimensions paramétré par $\vec{\theta}$

② Normalisation L2

On normalise chaque embedding $\vec{e}_i$.

normalisé $\vec{e}_i = \frac{\vec{e}_i}{\|\vec{e}_i\|_2}$

$\Rightarrow$ tous les $\vec{e}_i$ sont sur la même HYPER-PLANE UNITÉ

$\Rightarrow$ Évite que le vecteur "gagne" car il est plus long.

③ Mesure de SIMILARITÉ

a) Distance euclidienne normalisée

$d(\vec{e}_i, \vec{e}_j, \vec{\theta}) = \|\vec{e}_i - \vec{e}_j\|_2$

$\Rightarrow$ mesure la distance ℓ_2 embeddings normalisés

petit $d \Rightarrow$ plus proches

b) Coefficient de similarité

$d(\vec{e}_i, \vec{e}_j, \vec{\theta}) = \vec{e}_i^T \vec{e}_j$

= produit scalaire $\Rightarrow$ mesure la similarité

grand $d \Rightarrow$ plus proches

$\in [-1, 1]$: 1 vecteurs identiques

0 vect. $\perp$

-1 vect. pointant en direction opposée

DÉMO: de Dist. Eucl. Norm à Coeff. de similarité

$d = \|\vec{e}_i - \vec{e}_j\|_2 = \sqrt{(\vec{e}_i - \vec{e}_j)^T (\vec{e}_i - \vec{e}_j)}$

$= \sqrt{2 - 2 \vec{e}_i^T \vec{e}_j}$

$\Rightarrow$ d sera plus grand si $\vec{e}_i^T \vec{e}_j$ est petit

DONC:

L'idée est non plus d'apprendre la matrice M mais d'apprendre la fonction d'embedding

C'est le DEEP METRIC LEARNING (DML)

Le réseau $f(\vec{x}, \vec{\theta})$ est entraîné de telle manière:

- Si $\vec{x}_i \sim \vec{x}_j \Rightarrow \vec{e}_i \sim \vec{e}_j$

- Si $\vec{x}_i \not\sim \vec{x}_j \Rightarrow \vec{e}_i \not\sim \vec{e}_j$ (loignés)

DEEP METRIC LEARNING

3 techniques qui minimisent un certain "margin" pour s'assurer que les exemples similaires sont plus proches que les exemples non similaires.

La plupart de ces méthodes n'ont pas besoin de class labels.

1) Pairwise (CONTRASTIVE) lors d'un SIMILARITE metrique $(O(N^2))$

On cherche à minimiser la "Loss Contrastive"

$$L(\theta; \vec{x}_i, \vec{x}_j) = \frac{1}{2} \|y_i - y_j\| A(\vec{x}_i, \vec{x}_j)^2 + \|y_i - y_j\| \max[m - d(\vec{x}_i, \vec{x}_j), 0]^2$$

avec $y_i = y_j$ on minimise la distance (Perte quadratique)

longe loss (mise au carré) (Relle par ex)

Si $y_i \neq y_j$, on veut que la distance $d(\vec{x}_i, \vec{x}_j) \geq m$

→ Si oui → OK
Si non → on est sûr que la distance est suffisante pour séparer les points.

⇒ La CONTRASTIVE loss donne à :

- rapprocher les points similaires
- repousser les points différents (par trop. non cependant, juste au-delà d'un ϵ (de m))

Complexité temporelle : $O(N^2)$

on compare de chaque $\vec{x}_i$ avec tous les autres vecteurs (comme une double boucle "for")

2) TRIPLET loss $(O(N^3))$

Donc à : $L(\theta; \vec{x}_i, \vec{x}_i^+, \vec{x}_i^-) = \max \left[d(\vec{x}_i, \vec{x}_i^+) - d(\vec{x}_i, \vec{x}_i^-), 0 \right]$

Pour chaque exemple i (appelé l'ancre ou anchor) on trouve un exemple positif $\vec{x}_i^+$ et un exemple différent (négatif) $\vec{x}_i^-$ (alors différent)

⇒ Le triplet loss pénalise un cas où l'exemple positif $\vec{x}_i^+$ est plus éloigné que le négatif $\vec{x}_i^-$, ou cela d'un certain marge m .

→ "L'ancre $\vec{x}_i$ doit être plus proche de $\vec{x}_i^+$ que de $\vec{x}_i^-$ d'un margin m unités!"

Complexité temporelle : $O(N^3)$

Pairwise vs Triplet loss

PW

TRIPLET

- Optimisation des pos → points en 1ère indépendance de celle des négatives → marginales incompressibles
- Simple, moins de combinaisons $(O(N^2))$

- Limite le problème (+)

- Plus précis (+)

- Plus lent $(O(N^3))$ (-)

- Nécessite bonne stratégie de SAMPLING des bords $\vec{x}_i^-$ (travaux sur $\vec{x}_i^-$)

SUPPORT VECTORS MACHINES (ANDREW NG)

Contexte : Soit une régression logistique,

Rappel: log likelihood (à maximiser)

$$\ell(\theta) = \log L(\theta)$$

(pour l'ensemble des données)

$$= \sum_{i=1}^n y^{(i)} \log h(x^{(i)}) + (1 - y^{(i)}) \log (1 - h(x^{(i)}))$$

avec $L(\theta) = \prod_{i=1}^n \pi(y^{(i)} | x^{(i)}; \theta)$

(pour tous les échantillons)

$$= \prod_{i=1}^n \pi(y^{(i)} | x^{(i)}; \theta)$$

$$= \frac{1}{1 \dots 1} \left(h_\theta(x^{(1)}) \right)^{y^{(1)}} \left(1 - h_\theta(x^{(1)}) \right)^{1 - y^{(1)}}$$

avec $y^{(i)} = 0$ ou $1 \Rightarrow 1$ des 2 facteurs toujours égal à 1

c'est une écriture condensée:

$$\pi(y^{(i)} | x^{(i)}; \theta) = \begin{cases} \pi(y=1 | x^{(i)}; \theta) = h_\theta(x^{(i)}) \\ \pi(y=0 | x^{(i)}; \theta) = 1 - h_\theta(x^{(i)}) \end{cases}$$

fonction dite

avec $h_\theta(\vec{x}) = g(\theta^T \vec{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}$ val. comprise entre 0 et 1

et $\hat{y} = \begin{cases} 1 & \text{si } h_\theta(x) \geq 0.5 \\ 0 & \text{si } h_\theta(x) < 0.5 \end{cases}$ avec $z > 0$ ou $z < 0$

Mais la logit prédit bien une probabilité, servira à Q5

Rappel: Différence avec le PERCEPTRON

seul sigmoïde $g(z) = \begin{cases} 1 & \text{si } z \geq 0 \\ 0 & \text{si } z < 0 \end{cases}$

$\Rightarrow$ on se fait aucun calcul de probabilité, on prédit directement:

$$\hat{y} = \text{sign}(w^T x + b)$$

leaviside

supposons un training exemple $y=1$

Dans cette régression logistique, un point éloigné de la décision boundary aura:

- grand $\theta^T x \gg 0$
- grand $h_\theta(x) = \pi(y=1 | x; w, b) \approx 1$

Pareillement, un point $y=0$ très éloigné aura:

- $\theta^T x \ll 0$
- $h_\theta(x) \approx 0$

DONC: Pour un futur $\hat{y}$ (prédiction), soit $y=1$, nous serons plus sûrs de notre

prédiction si $\theta^T x \gg 0$. Idem pour un autre point $\hat{y}=0$ si $\theta^T x \ll 0$. Cette notion de "confiance" mène à la log. neg.

$\Rightarrow$ Voir CS 229, part V A.2 pour graphique et Murphy A.579-580

Notation: le modèle du SVM utilise une $g(z)$ non probabiliste ("HARD") comme la perception

$$h_{w,b}(x) = g(w^T x + b)$$

$$\text{avec } g(z) = \begin{cases} +1 & \text{si } z \geq 0 \\ -1 & \text{si } z < 0 \end{cases}$$

On remplace 0 par -1

on préfère travailler avec l'intercept séparément
 $\Rightarrow$ voir CS239 part V p.3

Marges: Fonctionnelle:

$$\hat{y}_i = y^{(i)} (w^T a + b)$$

pour 1 obs.

$$\hat{\gamma} = \min_{i=1..n} \hat{y}_i$$

pour le dataset de n obs.

voir CS239
 Part V, p.3,4,5

on prend la plus faible des $\hat{y}_i$, 1 pour tous les dataset

$$\gamma = \frac{\hat{\gamma}}{\|w\|}$$

Géométrique:

$$\gamma_i = y^{(i)} \left(\frac{w}{\|w\|} \right)^T a^{(i)} + \frac{b}{\|w\|}$$

$$\gamma = \min_{i=1..n} \gamma^{(i)}$$

pour le dataset

Optimal margin classifier: partie 1 = FORME PRIMALE (une distance, pas appelé ("functional"))

voir CS239
 Part V, p.6,7,8

$\Rightarrow$ Définition du problème d'optimisation et résoudre sur base du fait que on veut Maximiser la marge GÉOMÉTRIQUE, tout en se plaçant à la définition de la fonction objective GÉOMÉTRIQUE avec contraintes LINÉAIRES!

$\Rightarrow$ On cherche à attendre une fonction objective Quadratique Programming

Problème: En pratique, nous utilisons souvent du feature mapping

avec les SVM, dans ce cas on doit spécifier explicitement les KERNEL TRICKS (Nouveaux Kernels) parce on doit pouvoir exprimer notre objet d'intérêt en termes de vecteurs (car on veut de calculer explicitement les $\phi(x)$) (voir KERNELS D'INNER PRODUIT ou PERCEP Kernels)

$\Rightarrow$ On va chercher à écrire ces problèmes sous une forme DUAL (dual PRIMAL)

LAGRANGIEN

Fonction combinée que intègre : - la fonction objectif à maximiser / minimiser
- les contraintes via des multiplicateurs

Soit le problème d'optimisation suivant :

"Primal" $\left\{ \begin{array}{l} \min_w f(w) \\ s.t. h_i(w) = 0, i = 1, \dots, p \end{array} \right.$ "minimiser $f(w)$ sous les contraintes $h_i(w) = 0$

(avec au sein contraintes ici avec contraintes d'égalité mais sous contraintes d'inégalité)

Donc on veut trouver le w qui minimise une fonction, tout en respectant une ou plusieurs contraintes d'égalité (plus tard d'inégalités également)

Donc on veut que le minimum respecte les contraintes
[Donc on cherche une nouvelle fonction dont les dérivées s'annulent
au même point que les contraintes sont satisfaites.]
nouvelle fonction s'annule

⚠ Le minimum sous contraintes ne se trouve pas (facilement) là où le minimum de la fonction (sans) se trouve !

LAGRANGIEN (CAS SANS INTÉGRALITÉ)

① nombre de contraintes d'égalité

$$L(\vec{w}, \beta) = f(\vec{w}) + \sum_{i=1}^p \beta_i h_i(\vec{w})$$

Lagrange Multi. contrainte

f et h définies plus haut

La dérivée doit s'annuler :

(voir NON EXERCICE, LAGRANGIEN)

$$\nabla_{\vec{w}} L(\vec{w}, \beta) = \nabla f(\vec{w}) + \sum \beta_i \nabla h_i(\vec{w}) = 0$$

DONC

$$(A la solution optimale) \nabla_{\vec{w}} f(\vec{w}) = - \sum \beta_i \nabla h_i(\vec{w})$$

Les 2 gradients (de f et h_i) sont COLINÉAIRES (même direction, même intensité ou contraire)

Condition de stationnarité

⇒ donne les valeurs de $\vec{w}$ et de β

En pratique $\vec{w} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}$

On doit s'assurer de 3 choses :

① $\frac{\partial L}{\partial w_1} = 0$ et ② $\frac{\partial L}{\partial w_2} = 0$ donc $\nabla_{\vec{w}} L = \vec{0}$

MAIS AUSSI !

③ $\frac{\partial L}{\partial \beta} = 0$ Pourquoi ?

car $\frac{\partial L}{\partial \beta}$ retourne les contraintes "pures"

$$\frac{\partial L}{\partial \beta} = h(\vec{w}) \text{ et } g_i \text{ doit être respecté par définition}$$

C'est les (β_i) MULTIPLICATEURS de LAGRANGE, qui permettent de résoudre un problème avec contraintes.

Ensuite, en faisant varier ces multiplicateurs, on construisent la fonction DUALE
Et maximiser cette fonction duale, c'est le cœur de la DUALITÉ de LAGRANGE

EXEMPLE LAGRANGIEN #1 (CAS SANS INÉGALITÉ)

+ MURPHY L. 297
(exemple 8.5.1.1)

Soit le problème d'optimisation suivant.

OBJECTIF (quadratique) $f(x, y) = x^2 + 4y^2$

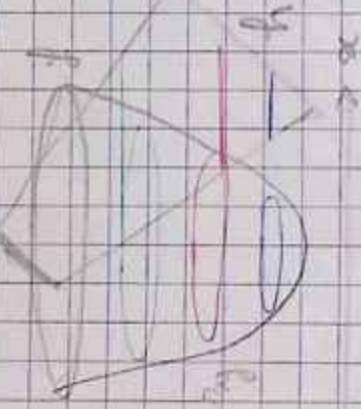
Contrainte (linéaire)

$$h(x, y) = x + 2y - 1 = 0$$

plan
lignes de niveau = droites //

paraboloïde elliptique
lignes de niveau (dans le plan (x, y)) = ellipses concentriques centrées en $(0, 0)$

Rem: Gradient est $\perp$ aux lignes de niveau (par def)



(une direction)

$$z = 1$$

$$z = 2$$

$$z = 3$$

→ Gradient de f en h
→ Gradient de f en h
→ Gradient de f en h
→ Gradient de f en h

la droite est tangente à l'ellipse au point où une droite tangente une ellipse coupe la trajectoire = optimum local (sans contrainte)



$$z = 1$$

$$z = 2$$

$$z = 3$$

point où les 2 quadratiques sont colinéaires (bon différent du minimum de la parabole)

C'est par là qu'on veut que le Lagrangien soit égal à zéro car c'est le minimum de cette parabole d'interaction (L'interaction doit être nulle, c'est à dire par une parabole)

$$\text{Rappel: } \nabla h(x, y) = \nabla h(w_1, w_2) = \begin{bmatrix} \frac{\partial h}{\partial w_1} \\ \frac{\partial h}{\partial w_2} \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \rightarrow \text{constant}$$

$$w_1 = w_1$$

$$w_2 = w_2$$

$$\nabla f(x, y) = \begin{bmatrix} 2x \\ 8y \end{bmatrix} \rightarrow \text{fonction de la fonction sur le plan } (x, y)$$

Rappel: Le gradient pointe dans la direction où f augmente le plus rapidement $\Rightarrow \perp$ aux courbes de niveau

* Remarque: par là, le gradient est une valeur constante en tous points (voir les numéros)

EXEMPLE LAGRANGIEN #2 (cas sans inégalité)

$$f(w_1, w_2) = w_1^2 + w_2^2$$

$$h(w_1, w_2) = w_1 + 2w_2 - 3 = 0$$

$$\text{Donc } L(w_1, w_2, \beta) = f(w_1, w_2) + \beta \cdot h(w_1, w_2) \\ = w_1^2 + w_2^2 + \beta(w_1 + 2w_2 - 3)$$

Dérivées partielles : si $\frac{\partial L}{\partial w} = 0$ alors chaque terme partiel de w composante de $\vec{w}$ s'annule

$$\frac{\partial L}{\partial w_1} = 2w_1 + \beta = 0 \Rightarrow w_1 = -\frac{\beta}{2}$$

$$\frac{\partial L}{\partial w_2} = 2w_2 + 2\beta = 0 \Rightarrow w_2 = -\beta$$

$$\frac{\partial L}{\partial \beta} = w_1 + 2w_2 - 3 = 0$$

Remplacez dans la contrainte :

$$\left(-\frac{\beta}{2}\right) + 2(-\beta) = 3$$

$$\Leftrightarrow -\frac{5\beta}{2} = 3 \quad \Leftrightarrow \beta = -\frac{6}{5}$$

Donc solution optimale :

$$w_1 = -\frac{\beta}{2} = \frac{3}{5}$$

$$w_2 = -\beta = \frac{6}{5}$$

LAGRANGIEN (note)

COMMENT PASSER DU PRIMAL AU DUAL

⚠ BTW, ne retourner pas forcément exactement les mêmes ne passe

Nous allons commencer par introduire la forme générale du Lagrangien (avec d'inégalité) contraintes

LAGRANGIEN (cas GÉNÉRAL)

$$L(\vec{w}, \vec{\alpha}, \vec{\beta}) = f(\vec{w}) + \sum_{i=1}^p \alpha_i g_i(\vec{w}) + \sum_{j=1}^q \beta_j h_j(\vec{w})$$

avec $\alpha_i \geq 0$

Contraintes d'égalité

en lien avec le problème d'optimisation suivant :

$$\text{PRIMAL} \begin{cases} \min_{\vec{w}} f(\vec{w}) \\ \text{s.t.} \quad g_i(\vec{w}) \leq 0, \quad i = 1, \dots, p \\ \quad \quad h_j(\vec{w}) = 0, \quad j = 1, \dots, q \end{cases}$$

Ici, α_i et β_j sont les **LAGRANGE MULTIPLIERS** (la notation de primal à dual peut se faire sans α_i , juste avec β_j btw)

Il y a 3 conditions = **CONDITIONS DE KARUSH - KUHN - TUCKER (KKT)**
(Sont les conditions d'optimalité)

① STATIONNARITÉ : gradient nul wrt $\vec{w}$

$$\nabla_{\vec{w}} L(\vec{w}, \vec{\beta}, \vec{\alpha}) = \vec{0}$$

② RESPECT DES CONTRAINTES

$$h_i(\vec{w}) = 0, \quad g_j(\vec{w}) \leq 0$$

③ COMPLÉMENTARITÉ

$$\alpha_j g_j(\vec{w}) = 0 \quad \forall j \quad \text{avec} \quad \alpha_j \geq 0 \quad \text{et} \quad g_j(\vec{w}) \leq 0$$

Donc si $g_j(\vec{w}) < 0$, alors $\alpha_j = 0$ (contrainte active)
si $g_j(\vec{w}) = 0$, alors $\alpha_j \geq 0$ (contrainte inactive)

Les problèmes d'optimisations sont donc des contraintes d'inégalité ne peuvent pas être résolus par calculs des dérivées partielles (sauf on s'est fait $\frac{\partial L}{\partial \alpha_i}, \frac{\partial L}{\partial \beta_j}$) et en les multipliant égaux à zéro

Pourquoi ? Car $\frac{\partial L}{\partial \alpha_i}$ ne va pas marcher ! Pourquoi ?

Imaginez un problème avec contraintes d'inégalité uniquement :

$$L(\vec{w}, \vec{\alpha}) = f(\vec{w}) + \sum_{j=1}^q \alpha_j g_j(\vec{w}) \quad \text{avec} \quad \alpha_j \geq 0$$

Ensuite, nous faisons $\frac{\partial L}{\partial \alpha_j} = 0$

$$\text{OR} \quad \frac{\partial L}{\partial \alpha_j} = g_j(\vec{w}) \quad \left(\text{comme} \quad \frac{\partial L}{\partial \beta_j} = h_j(\vec{w}) \right)$$

Donc cela revient à écrire $g_j(\vec{w}) = 0 \neq$ **contrainte d'inégalité**

⇒ A la place de résoudre le problème directement, on va chercher la **BORNE INFÉRIEURE** de la fonction objectif, en autorisant $\vec{w}$ à ne pas respecter les contraintes, mais en **penchant l'optimisation** via α et β sur $h_i(\vec{w}) \neq 0$, alors $f(\vec{w})$ peut être **strictement** > 0 , alors

- Si $h_1(\vec{w}) \neq 0$, alors $\beta, h_1(\vec{w})$ annule la relation
- Si $g_1(\vec{w}) > 0$, alors $\alpha, g_1(\vec{w})$ minimise l'inégalité

Donc on laisse $\vec{w}$ libre, mais on observe ce qui devient le Lagrangien quand on le minimise wrt $\vec{w}$, avec $\vec{\alpha}$ et $\vec{\beta}$ fixes.

C'est la définition de la Fonction DUALE.

$$g(\vec{\beta}, \vec{\alpha}) := \inf_{\vec{w}} L(\vec{w}, \vec{\beta}, \vec{\alpha})$$

INFIMUM, $\approx$ minimum, sauf que on exige pas qu'il soit atteint.

La fonction duale est utilisée si contraints d'inégalité car travailler sur la primalité se fait beaucoup plus.

Il est également primordial de se rendre compte de la nature de g :

C'est le même problème qu'au primal, contraints d'inégalité uniquement.

$$\begin{aligned} & \begin{cases} f(w) = w \\ g(w) = w - 1 \leq 0 \end{cases} \\ & L(w, \alpha) = f(w) + \alpha \cdot g(w) \\ & \quad = w^2 + \alpha(w - 1) \end{aligned}$$

Donc $g(\alpha) = \inf_w [w^2 + \alpha(w - 1)]$

On dérive pour trouver le minimum: $\frac{d}{dw} L(w, \alpha) = 2w + \alpha = 0 \Rightarrow w = -\frac{\alpha}{2}$

On injecte: $g(\alpha) = \left(-\frac{\alpha}{2}\right)^2 + \alpha\left(-\frac{\alpha}{2} - 1\right) = \frac{\alpha^2}{4} - \frac{\alpha^2}{2} - \alpha = -\frac{\alpha^2}{4} - \alpha$

CONCLUSION

$g(\alpha)$ est une parabole tournée vers le bas $\rightarrow$ concave

Remarque: le $w = -\frac{\alpha}{2}$ est souvent noté $w^*(\alpha) := \arg \min_w L(w, \alpha)$

C'est le w qui minimise L quand α est fixé.

LAGRANGIEN (Andrew Ng)

1. PARTIE PRIMAL pour un LAGRANGIEN

Reprenons juste dans la page PART V 1.8, après la définition du GENERALIZED LAGRANGIEN.

$\theta_D(\vec{w})$

On définit une fonction "qui détecte si $\vec{w}$ est admissible au sens" du PRIMAL

$$\theta_D(\vec{w}) = \max_{\alpha, \beta, \gamma \geq 0} L(\vec{w}, \vec{\alpha}, \vec{\beta})$$

Primal

$$\begin{cases} \min_{\vec{w}} f(\vec{w}) \\ \text{s.t. } g_i(\vec{w}) \leq 0, i=1, \dots, k \\ h_i(\vec{w}) = 0, i=1, \dots, l \end{cases}$$

CAS: 1) Soit $\vec{w}$ satisfait toutes les conditions

$$\text{Donc } \theta_D(\vec{w}) = \max_{\alpha, \beta, \gamma \geq 0} \left[f(\vec{w}) + \sum_{i=1}^k \alpha_i g_i(\vec{w}) + \sum_{i=1}^l \beta_i h_i(\vec{w}) \right]$$

≤ 0
($\alpha_i \geq 0$ et $g_i(\vec{w}) \leq 0$)
donc $\alpha_i g_i(\vec{w}) \leq 0$ $\Rightarrow$ $\vec{w}$ satisfait $\vec{w}$ du problème.)
 $= 0$

On en maximise au-dessus de $\vec{w}$ et ça nous donne la pénalité d'admissibilité. Si $\vec{w}$ est admissible, la pénalité est nulle.

Donc

$$= f(\vec{w})$$

2) Soit $\vec{w}$ n'est pas admissible :

Si $g_i(\vec{w}) > 0$: alors $\alpha_i g_i(\vec{w}) > 0$ et pour $\alpha_i \rightarrow \infty$, $\theta_D(\vec{w}) \rightarrow \infty$

Si $h_i(\vec{w}) \neq 0$: alors pour $\beta_i \rightarrow \infty$, $\theta_D(\vec{w}) \rightarrow \infty$

Donc

$$\theta_D(\vec{w}) = \begin{cases} f(\vec{w}) & \text{si } \vec{w} \text{ satisfait toutes les contraintes du PRIMAL} \\ +\infty & \text{sinon} \end{cases}$$

p^*

On définit ensuite la valeur optimale p^* de la fonction objectif :

$$p^* = \min_{\vec{w}} \theta_D(\vec{w}) = \min_{\vec{\alpha}, \vec{\beta}, \gamma \geq 0} L(\vec{w}, \vec{\alpha}, \vec{\beta})$$

C'est la VALEUR du problème PRIMAL